

10.5 ``Equivalence'' between Pointers and Arrays

There are a number of similarities between arrays and pointers in C. If you have an array

```
int a[10];
```

you can refer to `a[0]`, `a[1]`, `a[2]`, etc., or to `a[i]` where `i` is an `int`. If you declare a pointer variable `ip` and set it to point to the beginning of an array:

```
int *ip = &a[0];
```

you can refer to `*ip`, `*(ip+1)`, `*(ip+2)`, etc., or to `*(ip+i)` where `i` is an `int`.

There are also differences, of course. You cannot assign two arrays; the code

```
int a[10], b[10];  
a = b;                                /* WRONG */
```

is illegal. As we've seen, though, you *can* assign two pointer variables:

```
int *ip1, *ip2;  
ip1 = &a[0];  
ip2 = ip1;
```

Pointer assignment is straightforward; the pointer on the left is simply made to point wherever the pointer on the right does. We haven't copied the data pointed to (there's still just one copy, in the same place); we've just made two pointers point to that one place.

The similarities between arrays and pointers end up being quite useful, and in fact C builds on the similarities, leading to what is called ``the equivalence of arrays and pointers in C." When we speak of this ``equivalence" we do not mean that arrays and pointers are the same thing (they are in fact quite different), but rather that they can be used in related ways, and that certain operations may be used between them.

The first such operation is that it is possible to (apparently) assign an array to a pointer:

```
int a[10];  
int *ip;  
ip = a;
```

What can this mean? In that last assignment `ip = a`, aren't we mixing apples and oranges again? It turns out that we are not; C defines the result of this assignment to be that `ip` receives a pointer to the first element of `a`. In other words, it is as if you had written

```
ip = &a[0];
```

The second facet of the equivalence is that you can use the ``array subscripting" notation `[i]` on pointers, too. If you write

```
ip[3]
```

it is just as if you had written

```
*(ip + 3)
```

So when you have a pointer that points to a block of memory, such as an array or a part of an array, you can treat that pointer ``as if" it *were* an array, using the convenient `[i]` notation. In other words, at the beginning of this section when we talked about `*ip`, `*(ip+1)`, `*(ip+2)`, and `*(ip+i)`, we could have written `ip[0]`, `ip[1]`, `ip[2]`, and `ip[i]`. As we'll see, this can be quite useful (or at least convenient).

The third facet of the equivalence (which is actually a more general version of the first one we mentioned) is that *whenever* you mention the name of an array in a context where the ``value" of the array would be needed, C automatically generates a pointer to the first element of the array, as if you had written `&array[0]`. When you write something like

```
int a[10];
int *ip;
ip = a + 3;
```

it is as if you had written

```
ip = &a[0] + 3;
```

which (and you might like to convince yourself of this) gives the same result as if you had written

```
ip = &a[3];
```

For example, if the character array

```
char string[100];
```

contains some string, here is another way to find its length:

```
int len;
char *p;

for(p = string; *p != '\0'; p++)
    ;

len = p - string;
```

After the loop, `p` points to the `'\0'` terminating the string. The expression `p - string` is equivalent to `p - &string[0]`, and gives the length of the string. (Of course, we could also call `strlen`; in fact here we've essentially written another implementation of `strlen`.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail feedback](#)